# Arduino UNO Bluetooth Control Hub — HC-05 Multi-Device Guide

Oct 15, 2026 · @Vishal

All code for this guide is in the GitHub repository VishalRamKN/arduino-bluetooth-nodes; this guide's sketch is `combined-hub/BT_Hub_All/BT_Hub_All.ino` in the combined-hub folder. Students can clone or download it instead of copying code, and the repo's SETUP.md has the step-by-step setup.

## Overview

One Arduino sketch turns the UNO into a Bluetooth hub: you type a short text command in a Bluetooth terminal app on your phone, the HC-05 passes it to the UNO, and the UNO acts and sends a reply back to the phone.

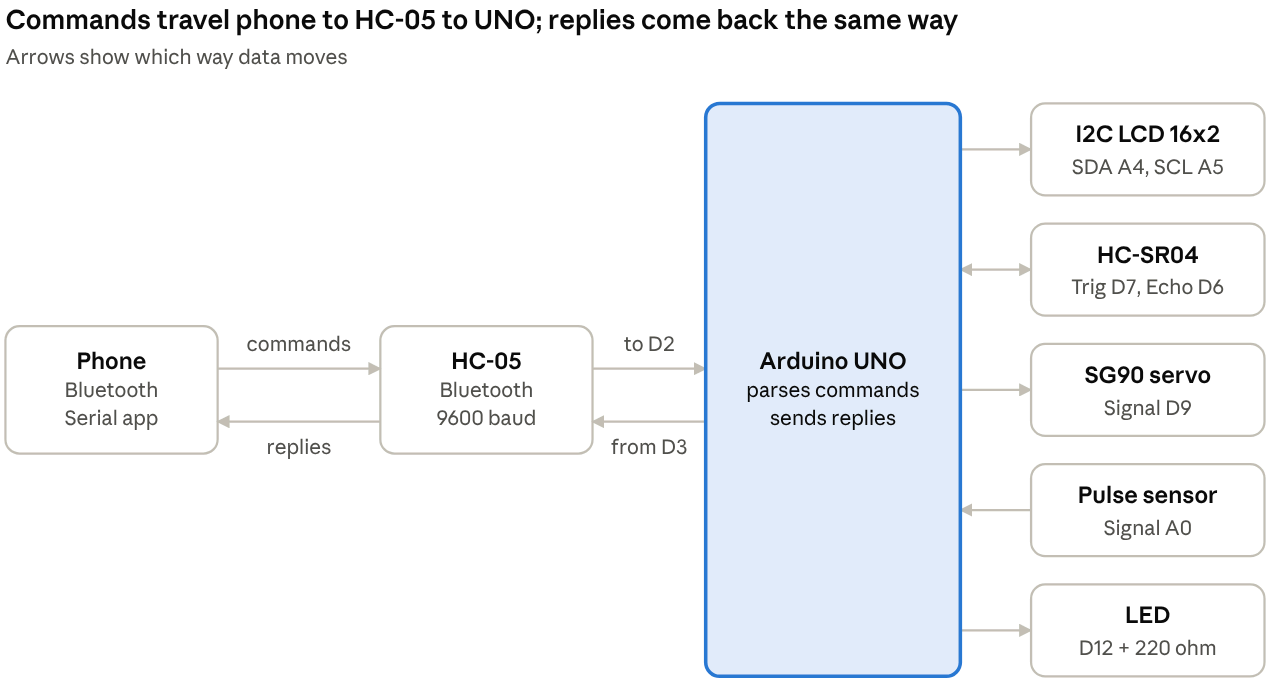
What each part does in this project:

- **I2C LCD (16x2)** shows any text you send with `LCD:<text>`.
- **HC-SR04 ultrasonic sensor** measures distance whenever you send `DIST`.
- **SG90 servo** turns to the angle you send with `SERVO:<0-180>`; `STATUS` or `SERVO?` reports its angle.
- **Heartbeat (pulse) sensor** reports BPM on `HR`, and with `HRON` it messages the phone on every detected beat.
- **LED** switches with `LED:ON` and `LED:OFF`.

Extra parts you need: a breadboard, jumper wires, one 1 kΩ and one 2 kΩ resistor (HC-05 voltage divider), one 220 Ω resistor (LED), and ideally a separate 5 V supply for the servo.

The HC-05 talks to the UNO on pins D2/D3 through SoftwareSerial, so the USB port (D0/D1) stays free. You can upload code without unplugging the Bluetooth module, and you can also type the same commands into the Arduino Serial Monitor for testing.

# System block diagram




system block diagram · phone, HC-05, UNO, 5 modules

The UNO sits in the middle: it only listens to the HC-05, and every module hangs off its pins. The ultrasonic sensor is two-way (trigger out, echo back) and the pulse sensor only sends data in.

# Pin table

Every module shares the UNO's 5V and GND; signals use D2, D3, D6, D7, D9, D12, A0, A4 and A5, leaving D0/D1 free for USB.

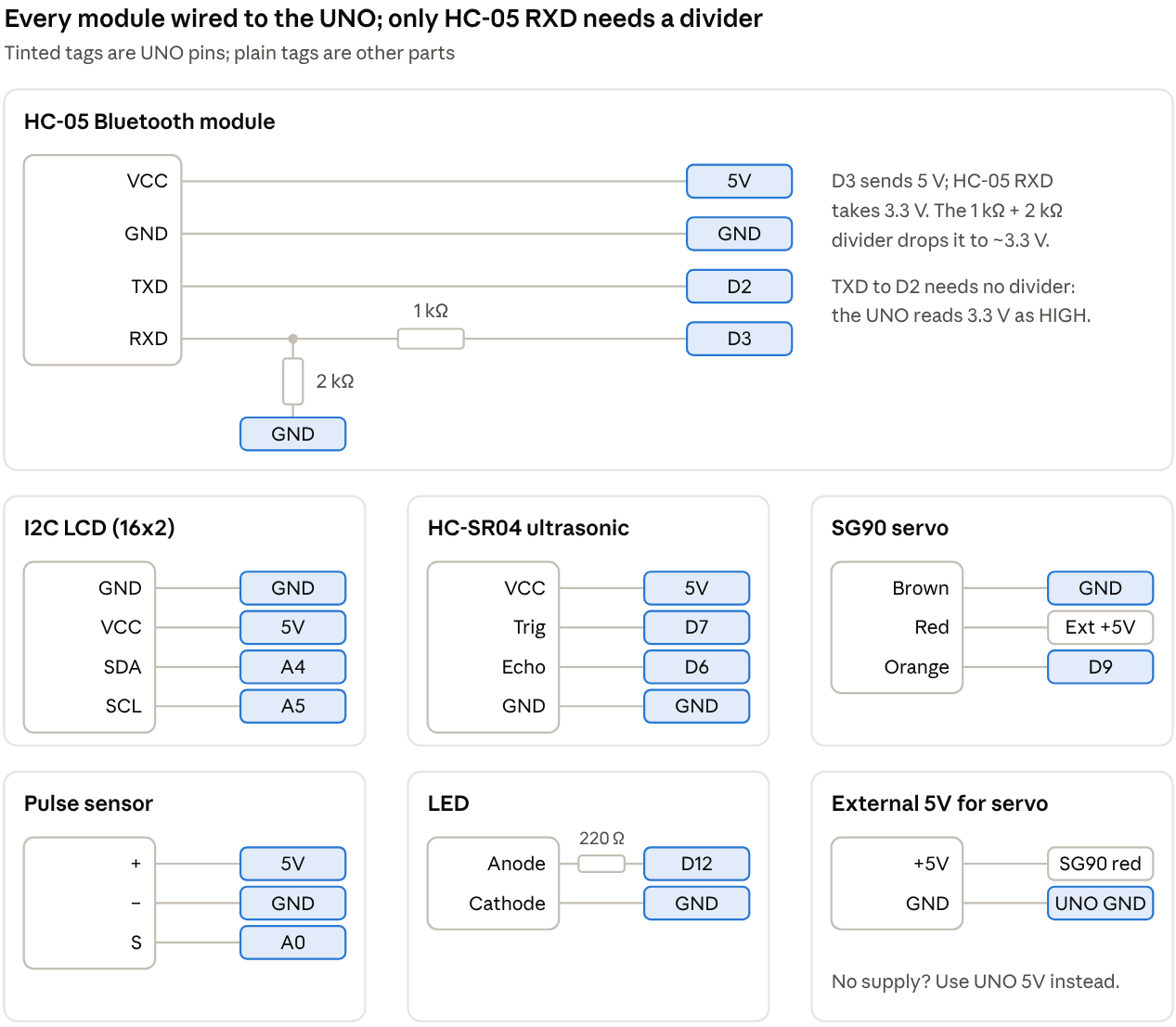
| Component | Component pin | UNO pin | Notes |
|---|---|---|---|
| HC-05 | VCC | 5V | The module board has its own 3.3 V regulator |
| HC-05 | GND | GND | |
| HC-05 | TXD | D2 | SoftwareSerial RX; direct wire, no resistor needed |

| Component | Component pin | UNO pin | Notes |
|---|---|---|---|
| HC-05 | RXD | D3 | Through a voltage divider: D3 → 1 kΩ → RXD, and RXD → 2 kΩ → GND |
| HC-05 | STATE, EN | Not connected | |
| I2C LCD | GND | GND | |
| I2C LCD | VCC | 5V | |
| I2C LCD | SDA | A4 | I2C data |
| I2C LCD | SCL | A5 | I2C clock |
| HC-SR04 | VCC | 5V | |
| HC-SR04 | Trig | D7 | Output from UNO |
| HC-SR04 | Echo | D6 | Input to UNO (5 V, safe for UNO) |
| HC-SR04 | GND | GND | |
| SG90 servo | Brown | GND | |
| SG90 servo | Red | 5V (external supply recommended) | Share GND with the UNO if external |
| SG90 servo | Orange | D9 | Control signal |
| Pulse sensor | + (VCC) | 5V | |
| Pulse sensor | − (GND) | GND | |
| Pulse sensor | S (signal) | A0 | Analog input |
| LED | Anode (long leg) | D12 | Through a 220 Ω resistor |
| LED | Cathode (short leg) | GND | |

## Power notes

- **HC-05 RXD is a 3.3 V input.** The UNO drives D3 at 5 V, so the 1 kΩ / 2 kΩ divider brings it down to 5 × 2 / (1 + 2) ≈ 3.3 V. HC-05 TXD outputs 3.3 V, which the UNO already reads as HIGH, so D2 needs nothing extra.
- **The servo is the power-hungry part.** An SG90 can pull several hundred mA when it moves, which can reset the UNO or drop the Bluetooth link on USB power. Powering it from a separate 5 V supply (1 A or more) with its GND tied to UNO GND avoids this. A 470 µF capacitor across the servo's 5V and GND also helps.
- **All grounds must connect together**, including any external supply's GND.

# Wiring diagram




wiring diagram · every module pin to its UNO pin

Each wire runs from the module pin on the left to the UNO pin in the tag. The only extra circuit is on HC-05 RXD: the 1 kΩ sits between D3 and RXD, and the 2 kΩ runs from RXD to GND. If you have no external supply, the servo's red wire can go to UNO 5V for light, unloaded use.

# Command reference

Type each command in the Bluetooth Serial app and press Send; commands are not case-sensitive, and the UNO answers every one.

| Module | Command | What it does | Example reply |
|---|---|---|---|
| LCD | `LCD:<text>` | Shows text on the LCD: first 16 characters on line 1, next 16 on line 2 | `LCD: shown` |
| LCD | `LCDCLR` | Clears the LCD | `LCD: cleared` |
| Ultrasonic | `DIST` | Measures distance once | `Distance: 23.4 cm` |
| Servo | `SERVO:<0-180>` | Turns the servo to that angle | `Servo moved to 90 deg` |
| Servo | `SERVO?` | Reports the current angle | `Servo is at 90 deg` |
| Heartbeat | `HR` | Reports the latest heart rate | `Heart rate: 74 BPM` |
| Heartbeat | `HRON` | Sends a message on every detected beat | `Beat! BPM: 74` |
| Heartbeat | `HROFF` | Stops beat messages | `Auto heartbeat: OFF` |
| LED | `LED:ON` | Turns the LED on | `LED: ON` |
| LED | `LED:OFF` | Turns the LED off | `LED: OFF` |
| All | `STATUS` | Servo angle, LED state, heart rate, auto mode | `Servo: 90 deg \| LED: OFF \| HR: 74 BPM \| Auto HR: OFF` |
| All | `HELP` | Lists all commands | Command list |

Examples: `LCD:Hello World`, `SERVO:45`, `led:on`. A wrong command gets `Unknown: <what you sent> (send HELP)`.

## Libraries and full sketch

Only one library needs installing; the rest ship with the Arduino IDE.

| Library | Where it comes from | Used for |
|---|---|---|
| LiquidCrystal I2C (by Frank de Brabander) | Library Manager: search "LiquidCrystal I2C" | LCD |

| Library | Where it comes from | Used for |
|---|---|---|
| Wire | Built in | I2C bus for the LCD |
| SoftwareSerial | Built in | HC-05 on D2/D3 |
| Servo | Built in | SG90 |

The heartbeat detection is written by hand rather than with a pulse-sensor library, so it cannot clash with the Servo library's timer.

**In the repo:** `combined-hub/BT_Hub_All/BT_Hub_All.ino` (combined-hub folder). The listing below is the same file, kept here for reading alongside the walkthrough.

```
/*
  Bluetooth Control Hub - Arduino UNO + HC-05
  Controls: I2C LCD, HC-SR04 ultrasonic, SG90 servo, analog pulse sensor, LED
  Every command ends with a newline. Send HELP for the list.
*/
#include <Wire.h>
#include <LiquidCrystal_I2C.h>
#include <SoftwareSerial.h>
#include <Servo.h>
#include <string.h>
#include <stdlib.h>

// ---------- Pin map ----------
const uint8_t BT_RX_PIN = 2;   // UNO receives on D2  <- HC-05 TXD
const uint8_t BT_TX_PIN = 3;   // UNO sends on D3     -> HC-05 RXD (via
divider)
const uint8_t ECHO_PIN  = 6;   // HC-SR04 Echo
const uint8_t TRIG_PIN  = 7;   // HC-SR04 Trig
const uint8_t SERVO_PIN = 9;   // SG90 orange wire
const uint8_t LED_PIN   = 12;  // LED anode through 220 ohm
const uint8_t PULSE_PIN = A0;  // Pulse sensor S pin

// ---------- Settings ----------
const uint8_t LCD_ADDR = 0x27; // change to 0x3F if the LCD stays blank
const long    BAUD     = 9600; // HC-05 default data-mode speed
const uint8_t CMD_MAX  = 64;   // longest command accepted
```

```
SoftwareSerial bt(BT_RX_PIN, BT_TX_PIN);
LiquidCrystal_I2C lcd(LCD_ADDR, 16, 2);
Servo servo;

// ---------- State ----------
int  servoAngle = 90;
bool ledOn      = false;
bool hrAuto     = false;

// One command buffer for Bluetooth, one for the USB Serial Monitor
char    btBuf[CMD_MAX + 1];
uint8_t btLen = 0;
char    usbBuf[CMD_MAX + 1];
uint8_t usbLen = 0;

// ---------- Heartbeat detection ----------
const unsigned long HR_SAMPLE_MS  = 10;   // 100 samples per second
const int           HR_MIN_SWING  = 40;   // smaller swing = no finger or
noise
const unsigned long HR_MIN_IBI    = 300;  // ignore beats faster than 200 BPM
const unsigned long HR_MAX_IBI    = 2000; // ignore gaps longer than 30 BPM
const unsigned long HR_TIMEOUT_MS = 3000; // no beat for 3 s = pulse lost

unsigned long hrLastSample = 0;
unsigned long hrLastBeat   = 0;
int           hrPeak = 512, hrTrough = 512;
bool          hrAbove = false;
unsigned long hrIbi[5];
uint8_t       hrIbiCount = 0, hrIbiIndex = 0;
int           bpm = 0;

// ---------- Replies go to the phone and to USB ----------
void reply(const char *msg) {
  bt.println(msg);
  Serial.println(msg);
}
void reply(const __FlashStringHelper *msg) {
  bt.println(msg);
  Serial.println(msg);
}

// ---------- 1. LCD ----------
void showOnLcd(const char *text) {
```

```
  lcd.clear();
  lcd.setCursor(0, 0);
  size_t n = strlen(text);
  for (size_t i = 0; i < n && i < 32; i++) {
    if (i == 16) lcd.setCursor(0, 1);   // wrap to line 2
    lcd.print(text[i]);
  }
}

// ---------- 2. Ultrasonic ----------
float readDistanceCm() {
  digitalWrite(TRIG_PIN, LOW);
  delayMicroseconds(2);
  digitalWrite(TRIG_PIN, HIGH);
  delayMicroseconds(10);              // 10 us trigger pulse
  digitalWrite(TRIG_PIN, LOW);
  unsigned long dur = pulseIn(ECHO_PIN, HIGH, 30000UL); // 30 ms timeout
  if (dur == 0) return -1.0;          // no echo = out of range
  return dur * 0.0343 / 2.0;          // speed of sound, there and back
}

// ---------- 3. Servo ----------
bool isNumber(const char *s) {
  if (*s == '\0' || strlen(s) > 3) return false;
  for (; *s; s++) {
    if (*s < '0' || *s > '9') return false;
  }
  return true;
}

void setServo(const char *arg) {
  while (*arg == ' ') arg++;
  if (!isNumber(arg) || atoi(arg) > 180) {
    reply(F("Error: use SERVO:0 to SERVO:180"));
    return;
  }
  servoAngle = atoi(arg);
  servo.write(servoAngle);
  char out[40];
  snprintf(out, sizeof(out), "Servo moved to %d deg", servoAngle);
  reply(out);
}
```

```
// ---------- 4. Heartbeat ----------
void updateHeartbeat() {
  unsigned long now = millis();
  if (now - hrLastSample < HR_SAMPLE_MS) return;
  hrLastSample = now;

  int v = analogRead(PULSE_PIN);

  // Envelope: jumps to new highs/lows, then slowly relaxes toward the signal
  hrPeak   = (v > hrPeak)   ? v : hrPeak - 1;
  hrTrough = (v < hrTrough) ? v : hrTrough + 1;
  int swing = hrPeak - hrTrough;
  int mid   = (hrPeak + hrTrough) / 2;
  int band  = swing / 8;              // hysteresis so noise can't double-
count

  if (swing < HR_MIN_SWING) {
    hrAbove = false;                  // too flat: no finger on the sensor
  } else if (!hrAbove && v > mid + band) {
    hrAbove = true;                   // rising edge = one beat
    unsigned long ibi = now - hrLastBeat;
    if (ibi >= HR_MIN_IBI) {
      hrLastBeat = now;
      if (ibi <= HR_MAX_IBI) {
        hrIbi[hrIbiIndex] = ibi;
        hrIbiIndex = (hrIbiIndex + 1) % 5;
        if (hrIbiCount < 5) hrIbiCount++;
        unsigned long sum = 0;
        for (uint8_t i = 0; i < hrIbiCount; i++) sum += hrIbi[i];
        bpm = 60000UL / (sum / hrIbiCount);
        if (hrAuto && hrIbiCount >= 2) {
          char out[32];
          snprintf(out, sizeof(out), "Beat! BPM: %d", bpm);
          reply(out);
        }
      }
    }
  } else if (hrAbove && v < mid - band) {
    hrAbove = false;                  // signal fell back: ready for next
beat
  }

  if (bpm > 0 && now - hrLastBeat > HR_TIMEOUT_MS) {
```

```
    bpm = 0;
    hrIbiCount = 0;
    hrIbiIndex = 0;
    if (hrAuto) reply(F("Pulse lost - rest finger on sensor"));
  }
}

void reportHeartRate() {
  if (bpm == 0) {
    reply(F("No pulse yet - rest finger on sensor for 5 s"));
    return;
  }
  char out[40];
  snprintf(out, sizeof(out), "Heart rate: %d BPM%s", bpm,
           hrIbiCount < 3 ? " (settling)" : "");
  reply(out);
}

// ---------- Status and help ----------
void reportStatus() {
  char out[72];
  snprintf(out, sizeof(out), "Servo: %d deg | LED: %s | HR: %d BPM | Auto HR:
%s",
           servoAngle, ledOn ? "ON" : "OFF", bpm, hrAuto ? "ON" : "OFF");
  reply(out);
}

void showHelp() {
  reply(F("Commands:"));
  reply(F("LCD:<text>     show text on LCD"));
  reply(F("LCDCLR         clear LCD"));
  reply(F("DIST           distance in cm"));
  reply(F("SERVO:<0-180>  move servo"));
  reply(F("SERVO?         servo angle"));
  reply(F("HR             heart rate now"));
  reply(F("HRON / HROFF   beat alerts on/off"));
  reply(F("LED:ON / LED:OFF"));
  reply(F("STATUS         everything"));
}

// ---------- Command dispatcher ----------
void handleCommand(char *cmd) {
  while (*cmd == ' ') cmd++;                      // trim front
```

```
  size_t n = strlen(cmd);
  while (n > 0 && cmd[n - 1] == ' ') cmd[--n] = '\0'; // trim back
  if (n == 0) return;

  char out[64];

  if (strncasecmp(cmd, "LCD:", 4) == 0) {
    const char *text = cmd + 4;
    while (*text == ' ') text++;
    showOnLcd(text);
    if (strlen(text) > 32) reply(F("LCD: shown (cut to 32 chars)"));
    else reply(F("LCD: shown"));
  } else if (strcasecmp(cmd, "LCDCLR") == 0) {
    lcd.clear();
    reply(F("LCD: cleared"));
  } else if (strcasecmp(cmd, "DIST") == 0) {
    float d = readDistanceCm();
    if (d < 0) {
      reply(F("Distance: out of range"));
    } else {
      char num[10];
      dtostrf(d, 0, 1, num);                      // float -> text, 1
decimal
      snprintf(out, sizeof(out), "Distance: %s cm", num);
      reply(out);
    }
  } else if (strncasecmp(cmd, "SERVO:", 6) == 0) {
    setServo(cmd + 6);
  } else if (strcasecmp(cmd, "SERVO?") == 0) {
    snprintf(out, sizeof(out), "Servo is at %d deg", servoAngle);
    reply(out);
  } else if (strcasecmp(cmd, "LED:ON") == 0) {
    ledOn = true;
    digitalWrite(LED_PIN, HIGH);
    reply(F("LED: ON"));
  } else if (strcasecmp(cmd, "LED:OFF") == 0) {
    ledOn = false;
    digitalWrite(LED_PIN, LOW);
    reply(F("LED: OFF"));
  } else if (strcasecmp(cmd, "HR") == 0) {
    reportHeartRate();
  } else if (strcasecmp(cmd, "HRON") == 0) {
    hrAuto = true;
```

```
    reply(F("Auto heartbeat: ON"));
  } else if (strcasecmp(cmd, "HROFF") == 0) {
    hrAuto = false;
    reply(F("Auto heartbeat: OFF"));
  } else if (strcasecmp(cmd, "STATUS") == 0) {
    reportStatus();
  } else if (strcasecmp(cmd, "HELP") == 0) {
    showHelp();
  } else {
    snprintf(out, sizeof(out), "Unknown: %s (send HELP)", cmd);
    reply(out);
  }
}

// ---------- Read a line from any serial port ----------
void pollStream(Stream &s, char *buf, uint8_t &len) {
  while (s.available()) {
    char c = s.read();
    if (c == '\n' || c == '\r') {              // end of command
      if (len > 0) {
        buf[len] = '\0';
        handleCommand(buf);
        len = 0;
      }
    } else if (len < CMD_MAX) {
      buf[len++] = c;
    }
  }
}

// ---------- Setup and main loop ----------
void setup() {
  Serial.begin(BAUD);
  bt.begin(BAUD);

  pinMode(TRIG_PIN, OUTPUT);
  pinMode(ECHO_PIN, INPUT);
  pinMode(LED_PIN, OUTPUT);
  digitalWrite(LED_PIN, LOW);

  servo.attach(SERVO_PIN);
  servo.write(servoAngle);                      // start at 90 deg
```

```
  lcd.init();
  lcd.backlight();
  showOnLcd("BT Hub ready    Send HELP");

  reply(F("BT Hub ready. Send HELP for commands."));
}

void loop() {
  pollStream(bt, btBuf, btLen);       // commands from the phone
  pollStream(Serial, usbBuf, usbLen); // commands from Serial Monitor
  updateHeartbeat();                  // runs all the time, never blocks
}
```

## How the code works

The main loop never waits: it checks both serial ports for a finished line, hands it to `handleCommand()`, and samples the pulse sensor every 10 ms in between.

### Command handling (shared)

- `pollStream()` collects characters until it sees Enter (`\r` or `\n`), then passes the whole line to `handleCommand()`. It runs once for Bluetooth and once for the USB Serial Monitor, so both work.
- `handleCommand()` trims spaces and compares the line with each command using `strcasecmp` / `strncasecmp`, which ignore upper/lower case.
- `reply()` prints every answer to both the phone and the Serial Monitor. Fixed messages use `F("...")`, which keeps them in flash memory and saves the UNO's 2 KB of RAM.

### 1. LCD

- `LCD:<text>` calls `showOnLcd()`, which clears the screen, prints characters 1–16 on line 1, moves the cursor to line 2, and prints characters 17–32. Anything longer is cut and the reply says so.
- `LCDCLR` simply calls `lcd.clear()`.
- At startup `lcd.init()` and `lcd.backlight()` switch the display on, and it shows "BT Hub ready / Send HELP".

## 2. Ultrasonic sensor

- `DIST` calls `readDistanceCm()`. It sends a 10 µs HIGH pulse on Trig; the sensor fires an ultrasonic burst and holds Echo HIGH until the sound returns.
- `pulseIn()` measures that HIGH time in microseconds. Sound travels 0.0343 cm/µs, and the time covers the trip there and back, so distance = time × 0.0343 / 2.
- If no echo returns within 30 ms (about 5 m), the reply is "out of range". The HC-SR04 is reliable from about 2 cm to 400 cm.

## 3. Servo

- `SERVO:<angle>` calls `setServo()`. `isNumber()` rejects anything that isn't 1–3 digits, and values above 180 are refused with an error message.
- A valid angle is stored in `servoAngle` and sent with `servo.write()`. The stored value is what `SERVO?` and `STATUS` report back.
- The servo starts at 90° on power-up.

## 4. Heartbeat sensor

- `updateHeartbeat()` reads A0 every 10 ms. It tracks the signal's recent peak and trough; their midpoint is the beat threshold, which adapts to your finger and the room light.
- Each time the signal rises above the threshold (plus a small hysteresis band) counts as one beat. The time since the previous beat is the inter-beat interval (IBI).
- Intervals shorter than 300 ms or longer than 2000 ms are thrown away as noise. The last 5 good intervals are averaged and BPM = 60000 / average IBI.
- If the swing between peak and trough is under 40, no finger is assumed. If no beat arrives for 3 s, BPM resets to 0.
- `HR` reports the current value on request. `HRON` sets `hrAuto`, and then every detected beat sends "Beat! BPM: xx" to the phone automatically; `HROFF` stops it.
- This is a hobby sensor: readings move with finger pressure and movement, and are not medically accurate.

## 5. LED

- `LED:ON` writes D12 HIGH and `LED:OFF` writes it LOW. The `ledOn` flag remembers the state so `STATUS` can report it.

### STATUS and HELP

- `STATUS` prints servo angle, LED state, heart rate and auto-heartbeat mode on one line.
- `HELP` prints the command list, so you never need this document on hand while using the app.

## Upload, phone setup and testing

Upload first, test over USB, then pair the phone; the HC-05 can stay wired the whole time because it does not use D0/D1.

1. **Install the library.** Arduino IDE → Tools → Manage Libraries → search "LiquidCrystal I2C" → install the one by Frank de Brabander.
2. **Wire everything** using the pin table and wiring diagram. Double-check the HC-05 RXD divider and that all grounds are joined.
3. **Upload.** Get the code with git clone https://github.com/VishalRamKN/arduino-bluetooth-nodes.git (or Code → Download ZIP on GitHub), open combined-hub/BT_Hub_All/BT_Hub_All.ino, set Board: Arduino Uno, pick the right port, and click Upload.
4. **Test over USB.** Open Serial Monitor at 9600 baud with line ending "Both NL & CR". You should see "BT Hub ready". Type `HELP`, `DIST`, `LED:ON` and check the replies.
5. **Pair the phone.** Power the UNO; the HC-05 LED blinks fast. On the phone go to Settings → Bluetooth, find "HC-05" and pair with PIN `1234` (some modules use `0000`). On Android 12 or newer, allow the app's "Nearby devices" permission.
6. **Set up the app.** In a Bluetooth terminal app such as "Serial Bluetooth Terminal" from the Play Store, open Devices → Bluetooth Classic and tap HC-05. When connected, the HC-05 LED changes to a slow double-blink.
7. **Set the line ending.** In the app's Settings → Send, choose newline "CR+LF" (or "LF"). Without a line ending the UNO never sees the end of the command.
8. **Optional: macro buttons.** Most terminal apps let you save buttons (M1, M2…) for `DIST`, `LED:ON`, `LED:OFF`, `HR` and `STATUS`, so you can tap instead of typing.

### Test sequence

| Send | Expect on phone | Expect on hardware |
|---|---|---|
| `HELP` | Command list | — |

| Send | Expect on phone | Expect on hardware |
|---|---|---|
| `LCD:Hello World` | `LCD: shown` | "Hello World" on the LCD |
| `DIST` | `Distance: xx.x cm` | Hold a hand 20 cm away; the number should be close to 20 |
| `SERVO:0`, then `SERVO:180` | `Servo moved to 0 deg`, `... 180 deg` | Servo sweeps end to end |
| `SERVO?` | `Servo is at 180 deg` | — |
| `LED:ON`, then `LED:OFF` | `LED: ON`, `LED: OFF` | LED lights, then goes off |
| `HR` (finger resting on sensor for 5–10 s) | `Heart rate: xx BPM` | — |
| `HRON` | `Beat! BPM: xx` roughly once a second | — |
| `STATUS` | One line with all states | — |

## Troubleshooting

Most problems come from the line-ending setting, the LCD address, or the servo browning out the supply.

| Symptom | Likely cause | Fix |
|---|---|---|
| Phone connects but nothing replies | App sends no line ending | Set newline to CR+LF in the app's Send settings |
| Garbled characters on the phone | Baud mismatch | HC-05 data mode default is 9600; if your module was changed, set `BAUD` in the code to match |
| UNO receives but phone gets nothing | HC-05 RXD not wired to D3, or divider wrong | Check D3 → 1 kΩ → RXD and RXD → 2 kΩ → GND |
| LCD backlight on but no text | Wrong I2C address or contrast | Try `LCD_ADDR = 0x3F`, or run the IDE's I2C scanner example; turn the blue contrast pot on the I2C board |

| Symptom | Likely cause | Fix |
|---|---|---|
| LCD shows blocks on line 1 only | Contrast too high or `lcd.init()` failed | Adjust the contrast pot; check SDA → A4, SCL → A5 |
| `Distance: out of range` every time | Trig/Echo swapped or nothing in front | Trig → D7, Echo → D6; aim at a flat object within 2–400 cm |
| UNO resets or Bluetooth drops when servo moves | Servo current sags the 5 V supply | Power the servo from a separate 5 V supply, join grounds, add a 470 µF capacitor |
| Servo twitches slightly while messages arrive | SoftwareSerial briefly pauses interrupts while receiving | Normal and minor; for none at all, move the HC-05 to D0/D1 (hardware serial) and unplug it while uploading |
| `HR` always says "No pulse yet" | Finger moving, pressing too hard, or signal too weak | Rest a fingertip lightly and still for 5–10 s; lower `HR_MIN_SWING` from 40 to 20 |
| BPM jumps around or reads double | Noise or room light reaching the sensor | Shade the sensor; raise `HR_MIN_SWING`; keep the finger still |
| Phone can't find HC-05 | Module not powered or already paired elsewhere | Check 5V/GND; HC-05 LED should blink fast when waiting for a connection |